DSA4050 Computer Vision Project 1

a) Project Description

Each student will design and implement a **computer vision application** that solves a practical real-world problem using **traditional image processing and object detection and classification techniques.** The use of deep learning models (e.g., CNNs, YOLO) is **not permitted**.

Students are free to choose any application domain, provided the project demonstrates the complete computer vision workflow from image acquisition to object detection and/or classification.

b) Learning Objectives

1. Acquire and prepare image data.
2. Apply image preprocessing techniques.
3. Segment objects of interest from images.
4. Extract meaningful image features.
5. Detect and/or classify objects using traditional machine learning methods/Haar-cascade methods.
6. Evaluate system performance using appropriate metrics.
7. Develop a working prototype demonstrating the solution

c) Suggested Application Areas

Students may choose any suitable application, including but not limited to:

- Agricultural inspection
- Medical image analysis
- Traffic monitoring
- Face recognition
- Attendance systems
- Industrial quality inspection
- Currency recognition
- Vehicle number plate recognition
- Document analysis
- Waste sorting
- Food quality inspection
- Wildlife monitoring
- Retail automation
- Security and surveillance
- Environmental monitoring
- Sports analytics

Note that you are free to choose an alternative project from any of the above.

d) Required Project Workflow

1. Problem Definition

Describe:

- The real-world problem.
- Motivation.
- Target users.
- Expected outputs.

2. Image Acquisition

Acquire images or video using:

- Webcam
- Smartphone camera
- Digital camera
- Public datasets
- Live video stream

3. Image Preprocessing

Apply suitable preprocessing techniques, such as:

- Image resizing
- Grayscale conversion
- Histogram equalization
- Noise reduction
- Contrast enhancement

Justify the preprocessing steps selected

4. Object Detection (Mandatory)

The project **must use Haar cascade classifiers** for object detection.

Possible implementation steps:

- Load a pre-trained Haar cascade classifier.
- Detect objects using a sliding window approach.
- Draw bounding boxes around detected objects.
- Handle multiple detections where appropriate.

You may use:

- Pre-trained Haar cascades available in OpenCV.
- Custom-trained Haar cascades (optional).

5. Feature Extraction (Optional, if Required)

Where the application requires further analysis after detection, students may extract features such as:

- Shape descriptors
- Color statistics
- Texture descriptors (e.g., GLCM, HOG, LBP)

These features may support subsequent classification or decision-making.

6. Classification

Following object detection, classify detected objects using traditional methods such as:

- Rule-based classification
- K-Nearest Neighbour (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Naïve Bayes

The classification stage should be relevant to the chosen application. For example:

- Face detected → Recognized/Unknown
- Vehicle detected → Car/Truck
- Fruit detected → Healthy/Defective

7. Performance Evaluation

Evaluate the developed system using appropriate metrics, including:

- Detection accuracy
- Precision
- Recall
- F1-score
- False positive rate
- Processing time

Discuss system strengths, limitations, and possible improvements.

8. Prototype Development

Develop a functional application with a simple interface (desktop, web, or command-line) that enables users to:

- Load an image or capture live video.
- Perform object detection.
- Display detection results using bounding boxes.
- Display classification results where applicable.

e) Recommended Technologies

- Python

- OpenCV
- NumPy
- scikit-learn /TensorFlow
- Matplotlib
- Joblib (for saving/loading classifiers, where applicable)

f) Deliverables

1. Source code.
2. Dataset (or a link to it).
3. Technical report.
4. Demonstration or presentation.

g) Report Structure

The report should include:

1. Introduction
2. Problem Statement
3. Objectives
4. Literature Review
5. Methodology
6. Implementation
7. Experimental Results
8. Discussion
9. Conclusion
10. Recommendations
11. References